

Study Guide for the FormalizedMathematics Cubical Project: Prerequisites, Literature, and a Fastest Path

The FormalizedMathematics Project

Last updated: 2026-07-11 (living document, updated as the project advances)

Abstract

This guide lists the knowledge needed to read and extend the project — a CCHM-style cubical type theory kernel implemented in Lean 4, plus a ~ 250 -theorem object-language library of synthetic homotopy theory and category theory — together with the books and papers to learn each piece from, and a compressed schedule for acquiring the whole stack as fast as is realistic.

Contents

1	The knowledge map	1
2	Annotated literature, by area	2
2.1	Dependent type theory	2
2.2	Homotopy type theory	2
2.3	Cubical type theory	2
2.4	Implementation	3
2.5	Category theory, higher categories, coherence	3
2.6	Algebraic topology	4
3	How the literature maps onto this repository	4
4	A fastest path	4

1 The knowledge map

The project sits at the intersection of five bodies of knowledge. The dependency structure is roughly:

$$\underbrace{\text{MLTT}}_{(1)} \longrightarrow \underbrace{\text{HoTT}}_{(2)} \longrightarrow \underbrace{\text{cubical TT}}_{(3)} \quad \underbrace{\text{implementation (NbE, checkers)}}_{(4)} \quad \underbrace{\text{category theory / topology}}_{(5)}$$

with (4) independent of (2)–(3) until they are combined in the kernel, and (5) providing the mathematical content of the library.

(1) Dependent type theory (MLTT)

Judgments, contexts, Π/Σ /identity types, universes, de Bruijn representation, substitution. You should be able to state the rules of intensional Martin-Löf type theory precisely and explain why the J eliminator does not prove UIP.

(2) Homotopy type theory

Paths as identities; homotopy levels (contractible/prop/set/groupoid); equivalences (contractible fibers, and why “isEquiv” must be a proposition); univalence; function extensionality; higher inductive types; the encode–decode method; truncations. The project’s library is essentially chapters 2–8 of the HoTT Book, cubically re-proved.

(3) Cubical type theory (CCHM)

The de Morgan interval; face lattices and partial elements; systems; `transp` and `hcomp` and how every HoTT construct (`funext`, J , univalence) becomes a *program*; Glue types and the univalence computation; composition for HITs. This is the single most project-specific body of knowledge.

(4) Implementation

Normalization by evaluation with defunctionalized closures; bidirectional type checking; how neutral terms carry annotations; Lean 4 as an implementation language (inductives, pattern matching, `partial` functions, the elaborator/interpreter/compiler distinction — the last of which this project has hard-won practical knowledge about).

(5) The mathematics being formalized

Fundamental group and covering spaces ($\pi_1(S^1)$, figure eight, free groups); classifying spaces $K(G, 1)$; basic category theory through 2-categories (functor categories, whiskering, interchange, coherence: Mac Lane’s pentagon); groups and quotients.

2 Annotated literature, by area

2.1 Dependent type theory

- **The HoTT Book, Chapter 1** (Univalent Foundations Program, *Homotopy Type Theory*, 2013; free online). Sufficient as an MLTT crash course if read actively.
- Egbert Rijke, *Introduction to Homotopy Type Theory* (arXiv:2212.11082). The cleanest modern textbook treatment of MLTT and HoTT together; recommended as the primary text.
- (Reference) Martin-Löf, *Intuitionistic Type Theory* (1984); Hofmann, *Syntax and Semantics of Dependent Types* (1997) for substitution/judgment rigor.

2.2 Homotopy type theory

- **The HoTT Book, Chapters 2–8**. Chapter 2 (paths), 3 (h-levels), 4 (equivalences), 6 (HITs), 7 (truncations), 8 ($\pi_1(S^1)$ and synthetic homotopy) map *directly* onto this project’s library modules.

- Licata–Shulman, *Calculating the fundamental group of the circle in homotopy type theory* (LICS 2013). The encode–decode method this project implements twice (S^1 and $K(G, 1)$).
- Rijke (op. cit.), Part II, for a more streamlined account.

2.3 Cubical type theory

- **Cohen–Coquand–Huber–Mörtberg**, *Cubical Type Theory: a constructive interpretation of the univalence axiom* (TYPES 2015/LIPIcs). The reference; this kernel implements its §§4–7 (including the §6.2 Glue transport, which the code cites by section number).
- Vezzosi–Mörtberg–Abel, *Cubical Agda: a dependently typed programming language with univalence and higher inductive types* (ICFP 2019) — the system whose library this project’s is closest to in spirit.
- Simon Huber’s PhD thesis, *Cubical Interpretations of Type Theory* (2016): canonicity and the operational reading of Kan operations.
- Mörtberg’s lecture notes *Cubical methods in homotopy type theory and univalent foundations* (EPIT 2022 spring school; and the associated `agda/cubical` exercises).
- (Context) Orton–Pitts, *Axioms for modelling cubical type theory in a topos*; Angiuli–Brunerie–Coquand–Harper–Hou–Licata for cartesian cubical variants — useful to understand design alternatives.

2.4 Implementation

- Andras Kovacs’ `elaboration-zoo` (GitHub): small, readable NbE elaborators; the best hands-on introduction to the evaluator architecture used here.
- Andreas Abel, *Normalization by Evaluation: Dependent Types and Impredicativity* (habilitation, 2013): the theory behind NbE.
- David Christiansen, *Checking Dependent Types with Normalization by Evaluation: A Tutorial* — gentle and directly relevant.
- *Theorem Proving in Lean 4* and *Functional Programming in Lean* (official, free): the metalanguage. For the elaborator/interpreter/compiler distinctions that bit this project, the Lean 4 metaprogramming book (Ullrich et al., community) is useful.
- `cubicaltt` (Coquand et al., GitHub) — a Haskell implementation of CCHM small enough to read end-to-end; the closest existing artifact to this project’s kernel.

2.5 Category theory, higher categories, coherence

- Riehl, *Category Theory in Context* (free PDF): all the 1-category theory used here.
- Mac Lane, *Categories for the Working Mathematician*: coherence (pentagon/triangle) chapters.
- Leinster, *Higher Operads, Higher Categories* (arXiv:math/0305049): definitions of weak n -categories and the comparison problem — background for the project’s stated goals.

- Ara, Maltsiniotis, Henry (papers on Grothendieck ∞ -groupoids) and Annenkov–Capriotti–Kraus–Sattler, *Two-level type theory and applications* — the precise statements of the open problems the project’s roadmap references.
- Van den Berg–Garner, *Types are weak ω -groupoids* (2011); Lumsdaine’s version — the meta-level tower the 2LTT direction would formalize.
- Licata–Finster, *Eilenberg–MacLane spaces in homotopy type theory* (LICS 2014) — the $K(G, 1)$ construction and its uniqueness; the project’s dimension-1 homotopy hypothesis ($K(\Omega A, 1) \equiv A$ for pointed connected 1-types) is the cubical, fully elaborated version of this circle of ideas.
- The HoTT Book §8.7 (van Kampen) and the Cubical Agda development of free groups — background for $\pi_1(S^1 \vee S^1) = F_2$ via reduced words and covering spaces; the project’s ‘LibWords’/‘helixF2’ follows the normal-form (reduced-word) strategy rather than the set-quotient one.
- For the untruncated coherence chapter (Eckmann–Hilton, Mac Lane pentagon/triangle in arbitrary types): HoTT Book §2.1–2.3 for the groupoid laws, and the ‘CubicalFoundations.GroupoidLaws’ module of the Cubical Agda library for the filler/connection proof style the project’s ‘squareExchange’/‘moving-middle idiom refines.

2.6 Algebraic topology

- Hatcher, *Algebraic Topology*, §§1.1–1.3 (free PDF): π_1 , covering spaces, van Kampen — the classical shadows of the library’s synthetic theorems ($\pi_1(S^1)$, figure-eight covers, F_2).
- May, *A Concise Course in Algebraic Topology*, for classifying spaces $K(G, 1)$.

3 How the literature maps onto this repository

Repository artifact	Primary reading
<code>Interval.lean</code> (DNF interval)	CCHM §2; Huber thesis
<code>Syntax.lean</code> / <code>TypeCheck.lean</code>	Christiansen tutorial; elaboration-zoo
<code>Semantics.lean</code> (NbE, Kan ops)	CCHM §§4–6; Abel habilitation
<code>Glue/ua/HCompU</code>	CCHM §6, §7.1; Cubical Agda paper §3
<code>LibCore–LibLoop</code> ($\pi_1(S^1)$)	HoTT Book ch. 2–3, 8.1; Licata–Shulman
<code>LibHedberg/h-levels</code>	HoTT Book ch. 3, 7; Rijke II
<code>LibCats</code> (functor cats, interchange)	Riehl; Mac Lane
<code>LibHITs</code> (susp/torus/pushout/trunc)	HoTT Book ch. 6; Cubical Agda lib
<code>LibQuot/LibWords</code> (F_2)	HoTT Book 6.10–6.11; Hatcher 1.3
<code>LibEM</code> ($K(G, 1)$, Codes)	Licata–Finster $K(G, n)$ (LICS 2014)
<code>LibCoherence</code> (pentagon)	Mac Lane VII; Leinster ch. 1
<code>HANDOFF.md</code>	the project’s own design-principle notes

4 A fastest path

The honest minimum for productive contribution is ~ 3 –5 months of focused part-time study for someone with general mathematical maturity and some functional programming; the plan below compresses aggressively by (a) running two tracks in parallel, (b) preferring *doing* over reading, and (c) using this repository itself as the final textbook.

Stage 0 (week 1): tooling

Install Lean 4 + VS Code; work through the first half of *Functional Programming in Lean*. Simultaneously install Agda + the `agda/cubical` library (used only as a sandbox).

Stage 1 (weeks 2–5): MLTT and HoTT core, by doing

Rijke chapters 1–12 *with all exercises*, or HoTT Book chapters 1–4 similarly. In parallel, in Cubical Agda: prove `funext`, Σ of contractibles contractible, Hedberg. Milestone: reprove `isPropsContr` without looking anything up.

Stage 2 (weeks 5–8): cubical structures

Read CCHM §§1–4 slowly, §§5–7 for architecture. In Cubical Agda, do the classics: `funExt` as a λ -swap, J from transport and connections, `trans` as an `hcomp`, and the $\pi_1(S^1) \cong \mathbb{Z}$ encode–decode *from scratch* (the single most valuable exercise for this project; budget a full week). Milestone: explain, on paper, why `transport(ua e)` computes to `e.fst` through Glue.

Stage 3 (weeks 8–11): implementation

Build a tiny dependent type checker with NbE following Christiansen / elaboration-zoo (untyped $\lambda \rightarrow$ dependent $\Pi \rightarrow$ universes). Then read this repository’s kernel in this order: `Interval` $\rightarrow$ `Syntax` $\rightarrow$ `Semantics` (eval/capp/force first, then `transp/hcomp`, then Glue last) $\rightarrow$ `TypeCheck`. Milestone: add a toy inductive (e.g. binary trees) end-to-end — syntax, values, eliminator, typing rule, and a library guard.

Stage 4 (weeks 11–14): the library and the frontier

Read `HANDOFF.md` (design principles and known pitfalls are the distilled experience of the project), then the library modules in chain order, checking each guard mentally against its HoTT Book counterpart. Milestone: port one small Cubical Agda lemma the library lacks, through the full workflow (Raw term, type, guard, native check if heavy).

Compression notes

If time is shorter still: Stage 1 can shrink to HoTT Book ch. 2 + Rijke exercises on equivalences only; Stage 3’s toy checker can be replaced by tracing this kernel’s `eval` on paper for three terms (`funExt`-application, `transp` on a constant family, `winding loop`). Categorical background (Riehl) and topology (Hatcher 1.1–1.3) can be acquired lazily, per-theorem, when reading the corresponding library modules. The items that *cannot* be skipped: the $\pi_1(S^1)$ exercise in Cubical Agda, and CCHM §4 (Kan operations) read until the composition rules feel inevitable.